

Experiment - 5

Design, Implementation, and Testing of Conditional Branching Programs in C

Question 1

Develop a C program that determines the roots of a quadratic equation in the form $ax^2 + bx + c = 0$. Read the coefficients a, b, and c, ensuring that $a \neq 0$, and compute the discriminant $D = b^2 - 4ac$.

Algorithm

1. Begin the program.
2. Read the values of coefficients a, b, and c.
3. Compute the discriminant using $D = b^2 - 4ac$.
4. When $D > 0$, compute and display two distinct real roots.
5. When $D = 0$, compute and display the two equal real roots.
6. When $D < 0$, compute and display the pair of complex roots.
7. End the program.

Test Case Table

Test Case	a	b	c	$D = b^2 - 4ac$	Condition	Expected Output
1	1	-5	6	1	$D > 0$	Roots = 3, 2
2	1	-4	4	0	$D = 0$	Roots = 2, 2
3	1	2	5	-16	$D < 0$	Roots = -1 + 2i, -1 - 2i

C Program

```
#include<stdio.h>
#include<math.h>

int main(void)
{
    int a,b,c;
    float D,R1,R2,IP,RP;

    printf("Enter the values of a,b and c:");
    scanf("%d%d%d",&a,&b,&c);

    if(a!=0)
    {
        D=b*b-4*a*c;
        printf("Discriminant=%.2f\n",D);

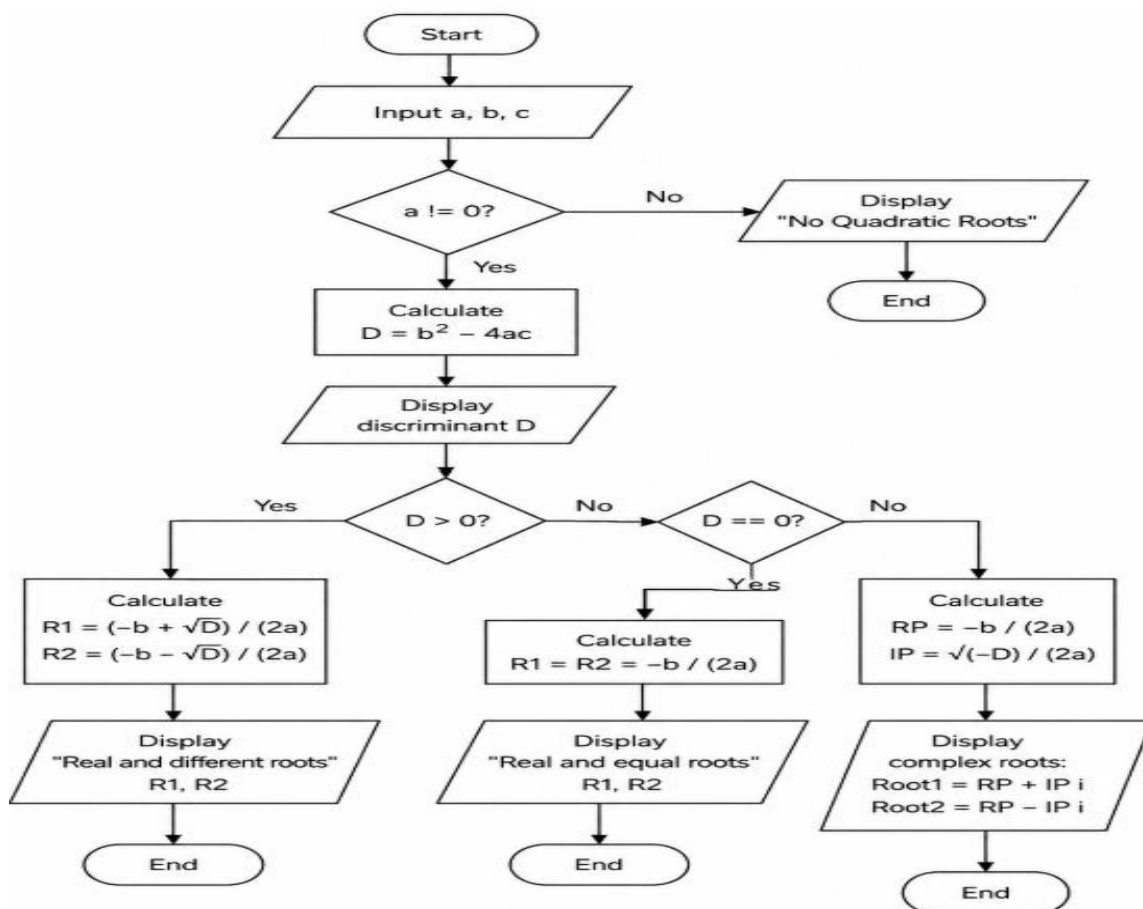
        if(D>0)
        {
            R1=(-b + sqrt(D))/(2.0*a);
            R2=(-b - sqrt(D))/(2.0*a);
            printf("It has real and different roots\n");
            printf("Root1=%.2f\n",R1);
            printf("Root2=%.2f\n",R2);
        }
        else if(D==0)
        {
            R1=R2=-b/(2.0*a);
            printf("Roots are equal: %.2f\n",R1);
        }
        else
        {
            IP=-b/(2.0*a);
            RP=sqrt(-D)/(2.0*a);
            printf("Roots are complex: %.2f + %.2fi, %.2f - %.2fi\n",IP,RP,IP,RP);
        }
    }
}
```

```

        R1=R2=-b/(2.0*a);
        printf("It has real and equal roots\n");
        printf("Root1=%.2f\n",R1);
        printf("Root2=%.2f\n",R2);
    }
    else
    {
        IP=sqrt(-D)/(2.0*a);
        RP=-b/(2.0*a);
        printf("Root1=%.2f + %.2fi\n",RP,IP);
        printf("Root2=%.2f - %.2fi\n",RP,IP);
    }
}
else
{
    printf("No Quadratic Roots\n");
}
return 0;
}

```

Flowchart



Compilation and Execution

```

(kali@kali)-[~/lab5]
$ gcc roots.c -o roots -lm

```

Execution - Test Case 1 (D > 0):

```
(kali@kali)-[~/lab5]
$ ./roots
Enter the values of a,b and c:1 -5 6
Discriminant=1.00
It has real and different roots
Root1=3.00
Root2=2.00
```

Execution - Test Case 2 ($D = 0$):

```
(kali@kali)-[~/lab5]
$ ./roots
Enter the values of a,b and c:1 -4 4
Discriminant=0.00
It has real and equal roots
Root1=2.00
Root2=2.00
```

Execution - Test Case 3 ($D < 0$):

```
(kali@kali)-[~/lab5]
$ ./roots
Enter the values of a,b and c:1 2 5
Discriminant=-16.00
Root1=-1.00 + 2.00i
Root2=-1.00 - 2.00i
```

Result:

The C program to find the roots of a quadratic equation using conditional statements was successfully implemented and executed. The program correctly determined real and distinct, real and equal, and complex roots based on the value of the discriminant.

Question 2

Create a menu-based C program for computing income tax according to India's old and new tax regimes. The program must take the taxpayer's name, age, gross income, and applicable deductions as input and then determine the final tax payable.

Algorithm

1. Begin the program.
2. Read the taxpayer name, age, and gross income.
3. Show the menu options: 1. Old Regime 2. New Regime.
4. Accept the selected menu option.
5. For choice 1, read all eligible deductions and cap every deduction at its permitted maximum.
6. Find total deductions and compute taxable income as gross income - total deductions. If this value is negative, use 0 as the taxable income.
7. For the old regime, calculate tax using the applicable age group and the taxable-income slabs specified in the experiment.
8. If old-regime taxable income is $\leq$ Rs. 5,00,000, apply a rebate equal to the smaller of Rs. 12,500 or the calculated tax; otherwise, the rebate is 0.
9. For choice 2, take taxable income as the gross income and calculate tax using the new-regime slabs specified in the experiment.
10. If new-regime taxable income is $\leq$ Rs. 12,00,000, apply a rebate equal to the smaller of Rs. 60,000 or the calculated tax; otherwise, the rebate is 0.
11. Determine tax after rebate by subtracting the rebate from the calculated tax.
12. Compute health and education cess as 4% of the tax remaining after rebate.
13. Obtain the final tax by adding the cess to the tax after rebate.
14. Display the taxpayer details, tax calculations, and final amount payable.
15. End the program.

Test Case Table

Test Case	Age	Gross Income	Regime	Deductions	Expected Final Tax
1	30	Rs. 8,00,000	Old	Rs. 1,50,000	Rs. 44,200
2	65	Rs. 9,00,000	Old	Rs. 1,50,000	Rs. 62,400
3	82	Rs. 9,00,000	Old	Rs. 1,50,000	Rs. 52,000
4	30	Rs. 10,00,000	New	-	Rs. 0
5	30	Rs. 15,00,000	New	-	Rs. 1,09,200

C Program

```
#include <stdio.h>

int main(void)
{
    char name[30];
    int age, choice;
    float income, taxable, tax = 0, rebate = 0;
    float tax_before_rebate;
    float d1, d2, d3, d4, d5, d6, d7;
    float total, cess, final_tax;

    printf("Enter name: ");
    scanf("%[^\n]", name);

    printf("Enter age: ");
    scanf("%d", &age);

    printf("Enter gross income: ");
    scanf("%f", &income);

    printf("\n1. Old Tax Regime");
    printf("\n2. New Tax Regime");
    printf("\nEnter choice: ");
    scanf("%d", &choice);

    if(choice == 1)
    {
        printf("Enter standard deduction: "); scanf("%f", &d1);
        printf("Enter savings deduction: "); scanf("%f", &d2);
        printf("Enter pension deduction: "); scanf("%f", &d3);
        printf("Enter medical deduction: "); scanf("%f", &d4);
        printf("Enter home loan interest: "); scanf("%f", &d5);
        printf("Enter savings account interest: "); scanf("%f", &d6);
        printf("Enter senior citizen interest: "); scanf("%f", &d7);

        if(d1 > 50000) d1 = 50000;
        if(d2 > 150000) d2 = 150000;
        if(d3 > 50000) d3 = 50000;
        if(d4 > 25000) d4 = 25000;
        if(d5 > 200000) d5 = 200000;
        if(d6 > 10000) d6 = 10000;
        if(d7 > 50000) d7 = 50000;

        total = d1+d2+d3+d4+d5+d6+d7;
        taxable = income-total;

        if(taxable < 0) taxable = 0;

        if(age < 60)
```

```

    {
        if(taxable <= 250000) tax = 0;
        else if(taxable <= 500000) tax = (taxable-250000)*0.05;
        else if(taxable <= 1000000) tax = 12500+(taxable-500000)*0.20;
        else tax = 112500+(taxable-1000000)*0.30;
    }
    else if(age < 80)
    {
        if(taxable <= 300000) tax = 0;
        else if(taxable <= 500000) tax = (taxable-300000)*0.05;
        else if(taxable <= 1000000) tax = 10000+(taxable-500000)*0.20;
        else tax = 110000+(taxable-1000000)*0.30;
    }
    else
    {
        if(taxable <= 500000) tax = 0;
        else if(taxable <= 1000000) tax = (taxable-500000)*0.20;
        else tax = 100000+(taxable-1000000)*0.30;
    }

    if(taxable <= 500000)
    {
        if(tax < 12500) rebate = tax;
        else rebate = 12500;
    }
}
else if(choice == 2)
{
    taxable = income;

    if(taxable <= 400000) tax = 0;
    else if(taxable <= 800000) tax = (taxable-400000)*0.05;
    else if(taxable <= 1200000) tax = 20000+(taxable-800000)*0.10;
    else if(taxable <= 1600000) tax = 60000+(taxable-1200000)*0.15;
    else if(taxable <= 2000000) tax = 120000+(taxable-1600000)*0.20;
    else if(taxable <= 2400000) tax = 200000+(taxable-2000000)*0.25;
    else tax = 300000+(taxable-2400000)*0.30;

    if(taxable <= 1200000)
    {
        if(tax < 60000) rebate = tax;
        else rebate = 60000;
    }
}
else
{
    printf("Invalid Choice\n");
    return 0;
}

tax_before_rebate = tax;
tax = tax-rebate;
cess = tax*0.04;
final_tax = tax+cess;

printf("\nName = %s", name);
printf("\nAge = %d", age);
printf("\nGross Income = %.2f", income);
printf("\nTaxable Income = %.2f", taxable);
printf("\nIncome Tax Before Rebate = %.2f", tax_before_rebate);
printf("\nRebate = %.2f", rebate);
printf("\nIncome Tax After Rebate = %.2f", tax);
printf("\nHealth and Education Cess = %.2f", cess);
printf("\nFinal Tax Payable = %.2f\n", final_tax);

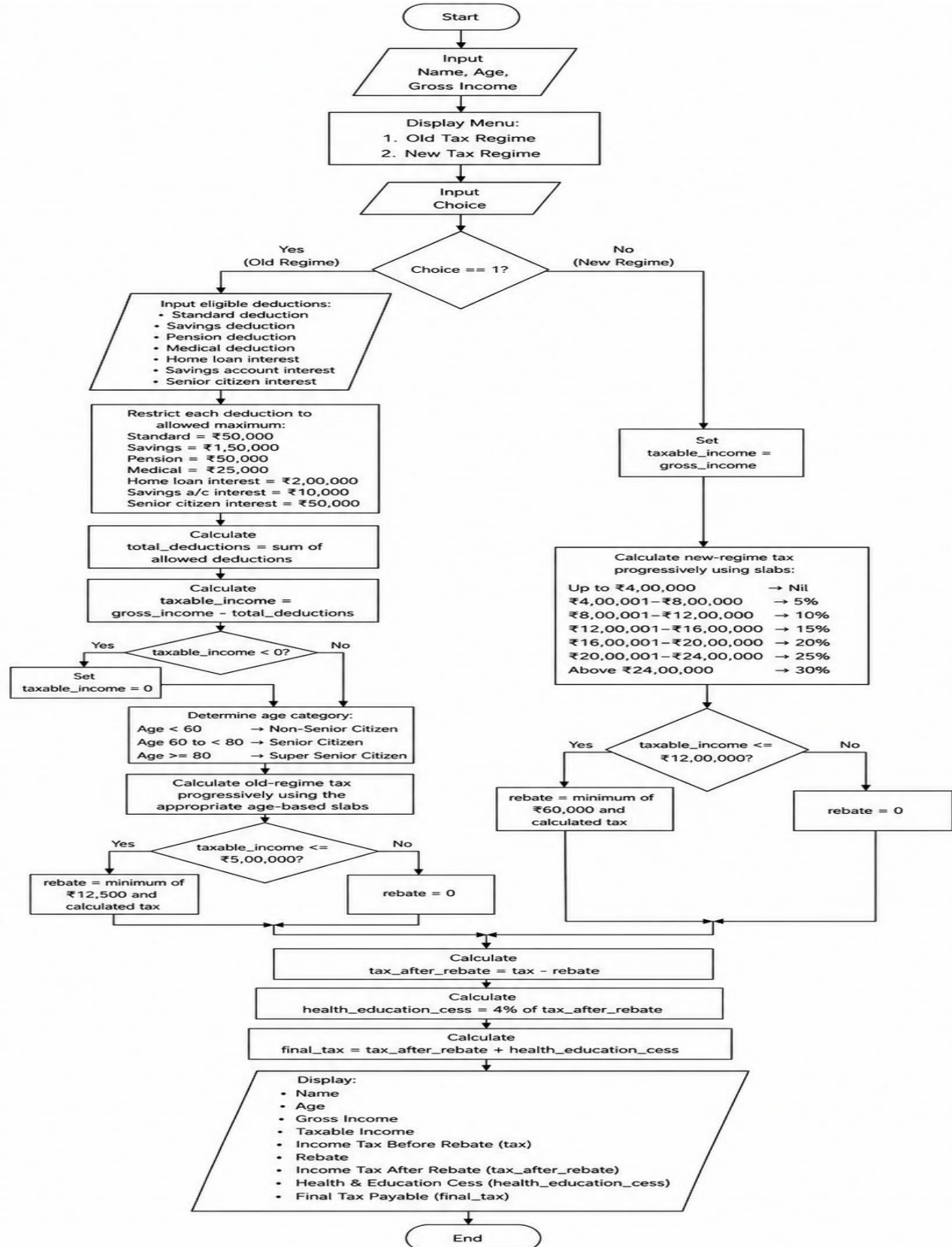
```

```

return 0;
}

```

Flowchart



Compilation and Execution

```
Trash  
(kali@kali)-[~/lab5]  
$ gcc tax.c -o tax -lm
```

Execution - Test Case 1 (Old Regime - Non-Senior Citizen):

```
(kali@kali)-[~/lab5]  
$ ./tax  
Enter name: Ritesh  
Enter age: 30  
Enter gross income: 800000  
  
1. Old Tax Regime  
2. New Tax Regime  
Enter choice: 1  
Enter standard deduction: 50000  
Enter savings deduction: 100000  
Enter pension deduction: 0  
Enter medical deduction: 0  
Enter home loan interest: 0  
Enter savings account interest: 0  
Enter senior citizen interest: 0  
  
——— TAX DETAILS ———  
Name = Ritesh  
Age = 30  
Gross Income = 800000.00  
Taxable Income = 650000.00  
Income Tax Before Rebate = 42500.00  
Rebate = 0.00  
Income Tax After Rebate = 42500.00  
Health and Education Cess = 1700.00  
Final Tax Payable = 44200.00
```

Execution - Test Case 2 (Old Regime -

Senior Citizen):

```
(kali@kali)-[~/lab5]  
$ ./tax  
Enter name: Ritesh  
Enter age: 65  
Enter gross income: 900000  
  
1. Old Tax Regime  
2. New Tax Regime  
Enter choice: 1  
Enter standard deduction: 50000  
Enter savings deduction: 100000  
Enter pension deduction: 0  
Enter medical deduction: 0  
Enter home loan interest: 0  
Enter savings account interest: 0  
Enter senior citizen interest: 0  
  
——— TAX DETAILS ———  
Name = Ritesh  
Age = 65  
Gross Income = 900000.00  
Taxable Income = 750000.00  
Income Tax Before Rebate = 60000.00  
Rebate = 0.00  
Income Tax After Rebate = 60000.00  
Health and Education Cess = 2400.00  
Final Tax Payable = 62400.00
```

Execution - Test Case 3 (Old Regime - Super Senior Citizen):

```
(kali㉿kali)-[~/lab5]
$ ./tax
Enter name: Ritesh
Enter age: 82
Enter gross income: 900000

1. Old Tax Regime
2. New Tax Regime
Enter choice: 1
Enter standard deduction: 50000
Enter savings deduction: 100000
Enter pension deduction: 0
Enter medical deduction: 0
Enter home loan interest: 0
Enter savings account interest: 0
Enter senior citizen interest: 0

——— TAX DETAILS ———
Name = Ritesh
Age = 82
Gross Income = 900000.00
Taxable Income = 750000.00
Income Tax Before Rebate = 50000.00
Rebate = 0.00
Income Tax After Rebate = 50000.00
Health and Education Cess = 2000.00
Final Tax Payable = 52000.00
```

Execution - Test Case 4 (New Regime - Rebate Case):

```
(kali㉿kali)-[~/lab5]
$ ./tax
Enter name: Ritesh
Enter age: 30
Enter gross income: 1000000

1. Old Tax Regime
2. New Tax Regime
Enter choice: 2

——— TAX DETAILS ———
Name = Ritesh
Age = 30
Gross Income = 1000000.00
Taxable Income = 1000000.00
Income Tax Before Rebate = 40000.00
Rebate = 40000.00
Income Tax After Rebate = 0.00
Health and Education Cess = 0.00
Final Tax Payable = 0.00
```

Execution - Test Case 5 (New Regime - Tax Payable Case):

```
(kali㉿kali)-[~/lab5]
$ ./tax
Enter name: Ritesh
Enter age: 30
Enter gross income: 1500000

1. Old Tax Regime
2. New Tax Regime
Enter choice: 2

—— TAX DETAILS ——
Name = Ritesh
Age = 30
Gross Income = 1500000.00
Taxable Income = 1500000.00
Income Tax Before Rebate = 105000.00
Rebate = 0.00
Income Tax After Rebate = 105000.00
Health and Education Cess = 4200.00
Final Tax Payable = 109200.00
```

Result:

The menu-driven C program to calculate income tax under the Old and New Tax Regimes was successfully implemented and executed. The program correctly calculated taxable income, rebate, health and education cess, and the final tax payable for the given test cases.